

ReCAT: Remember, Count, and Time: Structured Recurrent Memory for Robot Manipulation

Pankhuri Vanjani^{1*}, Mostafa Hatab^{1*}, Can Mizrakli^{1*}, Vaisakh Shaj², Zhuoyue Li¹,
Moritz Reuss³, Rudolf Lioutikov^{1,4}

¹Intuitive Robots Lab, Karlsruhe Institute of Technology (KIT), Germany ²University of Edinburgh, UK
³NVIDIA ⁴Robotics Institute Germany (RIG)

Abstract—Memory-dependent manipulation requires robots to make decisions using information that is no longer available to their current sensors, such as recalling an earlier visual cue, tracking task progress, counting repeated events, or estimating elapsed time. We present ReCAT, a language-conditioned policy with structured recurrent memory. An instruction-conditioned encoder forms features from the current observation. A recurrent memory integrates the observation stream through Mamba-2 layers and one causal attention layer. A flow-matching Transformer decoder reads the current and the historical representation through separate cross-attention in every block. ReCAT reaches 95.3% average success on LIBERO and 62.4% on RMBench, with the best or tied-best result on six of nine tasks. On three real-robot tasks probing spatial recall, event counting, and interval timing, the best ReCAT variant reaches 66.7% average success, against 8.3% for the strongest short-history baseline. Controlled comparisons within ReCAT show that the observation encoder and every-block memory conditioning are needed for this performance. They also show that update rules developed for efficient sequence modeling behave differently as robot memory: additive updates have the highest observed success on counting and timing, and delta-rule updates on spatial recall. Project website is at <https://intuitive-robots.github.io/ReCAT>

I. INTRODUCTION

Robot manipulation often requires information that is no longer available in the current observation. An object’s original location may have left the camera view, the number of completed repetitions may not be apparent, or the next action may depend on how much time has elapsed. In the PLANT task (Fig. 1), for example, the shovel-over-pot view looks alike after one scoop and after two, yet the required actions differ: scoop again, or put down the shovel and proceed to planting. Such tasks are non-Markovian with respect to the current observation, and the policy must combine current perceptual information with relevant interaction.

Policies obtain this history in different ways. Some use a window of recent observations. Some store selected observations in a memory bank [1], [2]. Others summarize the whole observation stream in a compact recurrent state [3]–[6]. Recent and concurrent work shows that such full-history recurrence helps in manipulation [7]–[12]. Yet, precise manipulation also requires the current spatial and proprioceptive

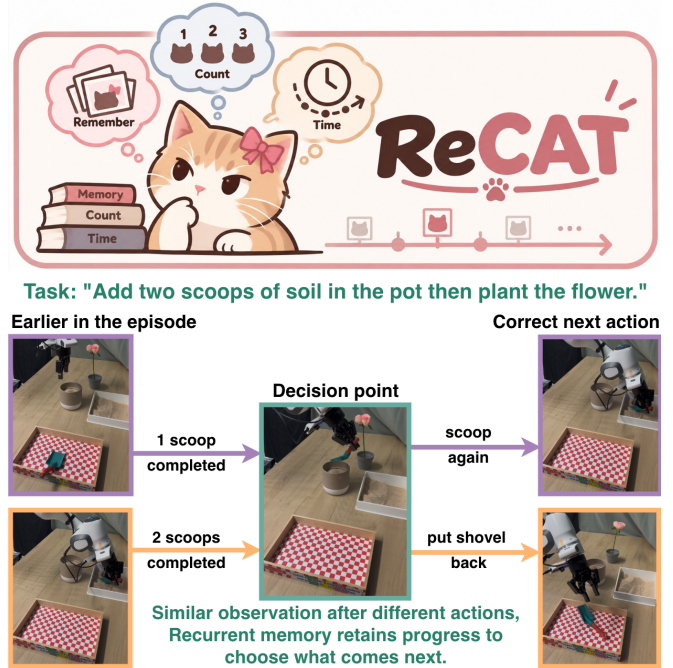


Fig. 1: Visually similar observations can require different actions depending on task history. ReCAT retains the completed scoop count in a compact recurrent memory, allowing the robot to scoop again after one scoop and put down the shovel after two.

observation. So, a policy needs both a history representation and direct access to the present.

We introduce ReCAT, a structured memory policy for language-conditioned manipulation. An instruction-conditioned encoder turns each observation into multimodal features. These features take two paths. One path feeds the action decoder directly. The other path updates a temporal memory. This memory is a stack of recurrent layers with one causal attention layer in the middle. A flow-matching Transformer decoder reads the current and the historical representation through separate cross-attention in every block. ReCAT therefore keeps a compact history representation and still sees the current observation directly when it generates actions.

*Equal contribution

We evaluate ReCAT on LIBERO, RMBench, and three real-robot tasks. The real-robot tasks probe spatial recall, event counting, and interval timing. ReCAT reaches 95.3% average success on LIBERO and 62.4% on RMBench. On the real-robot tasks it reaches 66.7%, compared with 8.3% for the strongest short-history baseline. Stage-wise evaluation shows where a policy fails: during manipulation, or when it must use history. Controlled comparisons within ReCAT then show how observation encoding, recurrent update rule, memory capacity, and decoder conditioning affect performance. Mamba variants reach the highest success on counting and timing tasks.

Our contributions are as follows:

- We present ReCAT, a structured policy whose temporal memory combines recurrent layers with a single causal attention layer. It conditions flow-matching action generation on separate current-observation and history pathways.
- ReCAT reaches 95.3% on LIBERO, 62.4% on RMBench with the best or tied-best result on six of nine tasks, and 66.7% across three real-robot memory tasks. It has the lowest measured inference latency among the evaluated memory-based policies.
- Controlled architectural and stage-wise comparisons show how observation encoding, recurrent update rule, memory capacity, and decoder conditioning affect immediate manipulation and later history-dependent decisions.

II. RELATED WORK

A. Memory in Visuomotor Imitation Learning

Robot policies differ in how they represent the past. Many keep past observations and attend to them. Fixed-window methods condition on recent visual tokens [13], [14]. Memory-bank methods store observations over longer horizons and retrieve them when needed [1], [15], [16]. These methods give direct access to earlier frames, as a result their attention cost grows with the number of retained tokens unless they select or compress. The second way carries a compact learned state through time. ReMem-VLA and μ VLA propagate recurrent queries or latent memory tokens [2], [12]. Gated Memory Policy adds a learned retrieval gate [17].

These policies show that a compact state can carry task history. Each of them, however, pairs one memory mechanism with one way of feeding it to the policy. This leaves open a question: *how does the recurrent update itself shape what is remembered, and how should that memory be combined with the current observation for action generation?* ReCAT is built around both parts of it. Its memory is a stack of recurrent layers with one causal attention layer, and the update rule inside that stack is interchangeable. It’s flow-matching decoder attends to the current observation and to the memory through separate cross-attention in every block. Section IV evaluates this design and compares update rules, capacities within it.

B. Recurrent Memory Architectures for Robot Manipulation

State-space models (SSMs) and related recurrent architectures have been used both as action-generation backbones and as history encoders in robot learning. MaIL uses a

Mamba encoder-decoder for behavior cloning, Mamba Policy combines Mamba and attention within a 3D diffusion policy, and DiSPo applies an SSM across discretized action scales [18]–[20]. AnoleVLA uses causal Mamba recurrence to fuse proprioceptive, visual, and language features for action chunk prediction [21]. For temporal conditioning, the Mamba motion encoder extracts temporal features, MTIL processes full trajectories with Mamba-2, RoboSSM processes long in-context demonstrations, and Embodied-SlotSSM maintains object-centric memory [7]–[9]. Concurrent work conditions on the full observation history as well. DSSP pairs a Mamba history representation with a dynamics-aware objective and hierarchical prefix conditioning for an SSM diffusion denoiser [10]. Chronos combines a selective historical state with a physics-informed action prior [11]. RoboTTT adapts fast weights at test time with Gated DeltaNet recurrence [22]. These policies each fix one recurrent backbone and one route from memory to action. ReCAT differs on both counts. Its temporal memory mixes recurrent layers with causal attention, and its action decoder receives the memory and the current observation through separate cross-attention rather than as a prefix or a prior.

These approaches draw on several families of linear recurrence. All keep a fixed-size state at constant per-step cost [23]–[25]. Additive memories, such as linear attention and Mamba-2, accumulate key-value writes under a learned decay [23], [25], [26]. Delta-rule models erase and rewrite information along matching key directions [27], [28]. A third family adds richer state dynamics through negative eigenvalues, reflections, or complex-valued rotations [5], [29]. All of them can be written as $S_t = A_t S_{t-1} + b_t$. The structure of A_t sets the memory semantics: accumulation, replacement, or rotation. Pure recurrence has a known limit, though. A fixed-size state struggles with exact recall of earlier tokens [30]. Efficient long-context language models therefore mix a few attention layers into the recurrent stack, and this hybrid design is now common [25], [31]. Mamba Policy brings the same idea to a robot policy backbone [19]. ReCAT uses this hybrid design in two places. Its observation encoder mixes SSM and attention layers over the tokens of the current frame. Its temporal memory mixes them across time, with one causal attention layer among the recurrent layers, and the recurrent update rule within that stack is interchangeable. The two parts are meant to complement each other: the recurrent layers track task progress at constant cost, while the attention layer can look back at a specific earlier observation.

III. METHODOLOGY

A. Problem Formulation

We consider language-conditioned imitation learning for partially observed robot manipulation. The training set consists of demonstrations $\mathcal{D} = (\ell^{(i)}, (s_t^{(i)}, a_t^{(i)})_{t=1}^{T_i})_{i=1}^N$, where $\ell^{(i)}$ is the task instruction, $s_t^{(i)} = (v_t^{(i)}, p_t^{(i)})$ combines the camera observations and proprioception, and $a_t^{(i)} \in \mathbb{R}^{d_a}$ is the demonstrated action. We denote the available observation history by $\mathcal{H}_t = (s_{1:t}, \ell)$ and an action chunk of length K by $A_t = (a_t, \dots, a_{t+K-1})$.

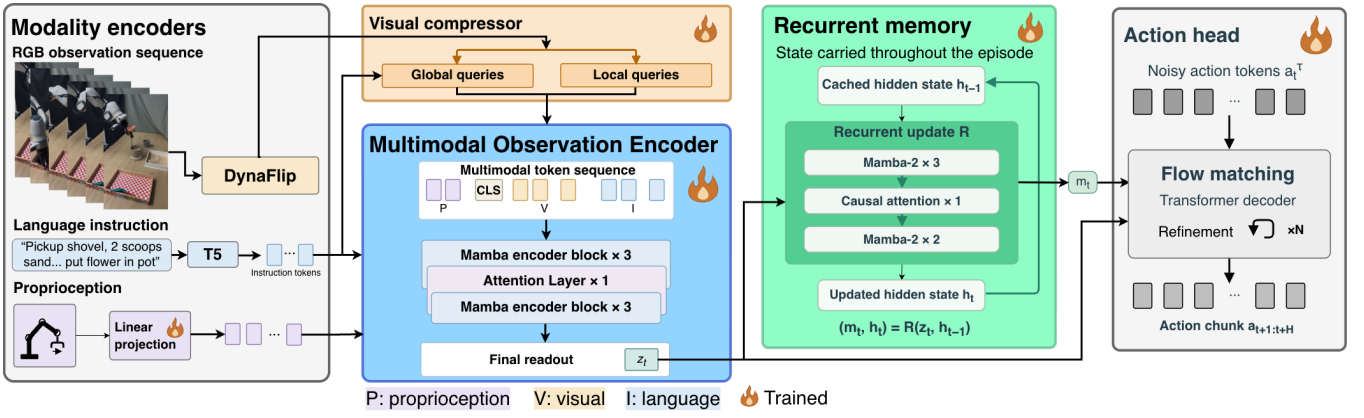


Fig. 2: Overview of ReCAT. Frozen DynaFLIP and T5 encoders (LoRA-adapted) feed a language-conditioned visual compressor and a six-layer Mamba–attention frame encoder, which produces one feature vector z_t per step. The same z_t feeds the action decoder directly and is the input to the recurrent memory, which returns the readout m_t . The flow-matching decoder attends to z_t and m_t through separate cross-attention in every block. The decoder backbone is shared across all experiments. The ablations vary the frame-encoder mixer, the recurrent update rule, memory depth and width, and how m_t enters the decoder.

In memory-dependent tasks, the current observation s_t may be insufficient to determine the task state, so visually similar observations can require different actions depending on prior events. The policy is therefore non-Markovian with respect to s_t alone and is modeled as $\pi_\theta(A_t | \mathcal{H}_t)$. Rather than storing the growing history $\mathcal{H}_t$, ReCAT updates a compact recurrent representation as new observations arrive.

B. ReCAT

ReCAT has three parts (Fig. 2). An instruction-conditioned observation encoder turns the current images, instruction, and proprioception into one feature vector z_t . A temporal memory takes z_t as its input at every step, updates its retained state h_t , and returns a history-conditioned readout m_t . A flow-matching action decoder predicts the next action chunk from z_t and m_t . The same z_t therefore feeds the decoder directly and drives the memory.

a) Current-Observation Encoding: DynaFLIP [32] encodes each camera image into a 16×16 patch grid P_t . Its T5 text encoder produces instruction tokens L . Both backbones stay frozen and are adapted with LoRA. Following Compressor-VLA [33], we compress each patch grid with language-conditioned global queries and local window queries. The global queries keep task-relevant context. The local queries keep fine spatial detail.

$$Z_t = \text{CrossAttn}(\gamma(\bar{\ell}) \odot Q + \beta(\bar{\ell}), P_t), \quad (1)$$

Here $\bar{\ell}$ summarizes the instruction, Q are the learned global queries, and γ, β are FiLM parameters. The local window queries work the same way over their windows. Global and local outputs are concatenated into 80 tokens per camera. A six-layer encoder then mixes these tokens with the instruction tokens and a proprioception token within the frame. Five of its layers are Mamba-2 and the fourth is bidirectional attention. This encoder carries no state across timesteps. Its last position is read out as the frame feature $z_t \in \mathbb{R}^{768}$.

b) Recurrent Memory: The recurrent memory summarizes the whole observation history. It is a stack of six blocks of width 768. Five blocks are Mamba-2 layers and have no feed-forward network (FFN). The fourth block is a causal attention layer and has one FFN. At every step the memory takes z_t as input, updates its retained state, and returns a readout,

$$(q_t, m_t) = R_\phi(z_t, q_{t-1}), \quad (2)$$

where $q_t = (h_t, C_t)$. Here h_t denotes the persistent recurrent states of the five Mamba-2 layers, and C_t is the key-value cache of the attention layer. The size of h_t is fixed by the architecture. The cache C_t holds one entry per step and therefore grows linearly with episode length. Each entry is one 768-dimensional vector, so the memory never stores images or patch tokens. All of q_t is reset at the start of each episode. The readout m_t is the normalized output of the last block, not the state itself.

Two properties of the memory are set by configuration. The first is capacity: the number of blocks and the state width d_{state} of each Mamba-2 layer. The second is the update rule of the recurrent layers. Mamba-2 is the default. For comparison we replace it with Mamba-3 or Gated DeltaNet-2 and keep the observation encoder, the attention layer, the action decoder, and the training recipe unchanged. We match parameter counts across update rules. Table I summarizes what each rule does to information already in the state. During training, the recurrence runs over the whole trajectory as a parallel scan. At inference, it runs one step at a time.

c) Observation and memory conditioned Action Generation: Following flow-matching based policies [34]–[36], a Transformer decoder predicts an action chunk $A_t \in \mathbb{R}^{K \times d_a}$ with $K = 16$. For Gaussian noise X_0 and $\tau \sim \mathcal{U}[0, 1]$, we set $X_\tau = (1 - \tau)X_0 + \tau A_t$ and minimize

TABLE I: Update rules of the recurrent layers compared in Sec. IV. Each step writes a cue-content pair (k_t, v_t) derived from z_t into the state h_t . The rules differ in what the write does to information already stored. The descriptions give the intended semantics of each rule. Whether a trained policy uses them this way is an empirical question.

Model	State update	Update semantics
Mamba-2	$\text{diag}(a_t)h_{t-1} + v_t k_t^\top$	Accumulate: storing a similar cue again adds another write to its trace, so repeated occurrences of an event build up in the state, the property counting requires.
Mamba-3	$\text{diag}(a_t e^{i\theta_t})h_{t-1} + v_t k_t^\top$	Accumulate + rotate: as above, with phase evolution that provides an additional signal for temporal progression or motion.
GDN-2	$\alpha_t(I - \beta_t k_t k_t^\top)h_{t-1} + \beta_t v_t k_t^\top$	Replace: storing a similar cue overwrites its previous content with the latest value, while other stored cues remain undisturbed. Repeated identical events therefore leave a single trace.

$$\mathcal{L}_{\text{FM}} = \mathbb{E} \left[\|v_\theta(X_\tau, \tau; z_t, m_t) - (A_t - X_0)\|_2^2 \right]. \quad (3)$$

The decoder has four blocks. Each block applies self-attention over the action tokens, then cross-attention to z_t , then cross-attention to m_t , then a feed-forward network. The two cross-attention operations have separate parameters, so the current observation and the memory enter every block through separate paths. Flow time τ conditions every block through zero-initialized adaptive layer normalization.

d) Implementation and training: Training processes each demonstration as one causal scan over the full episode. The vision and text backbones stay frozen and are adapted with LoRA of rank 16 and 8, respectively. Real-robot policies train for 500 epochs. At inference, the decoder integrates the velocity field with $N = 4$ Euler steps. The robot executes the first $k = 4$ actions of each chunk and then replans. The frame encoder runs and the memory advances at every control step, including during chunk execution. The memory’s recurrent states and attention cache are reset between episodes.

IV. EVALUATION

We evaluate ReCAT on two simulation benchmarks and three real-robot tasks. The evaluation asks three questions:

- **(RQ1)** How effective is ReCAT on standard and on memory-dependent manipulation?
- **(RQ2)** How do its observation encoder and its memory conditioning affect performance?
- **(RQ3)** How do the recurrent update rule and the memory capacity affect different memory demands?

We report stage-wise behavior within these sections and computational cost separately.

A. Experimental setup

a) Simulation benchmarks: We evaluate ReCAT on LIBERO [37], including Spatial, Object, Goal, and LIBERO-10, using two cameras and 50 epochs. Each 10-task suite is evaluated over 500 episodes (50 per task). Since LIBERO

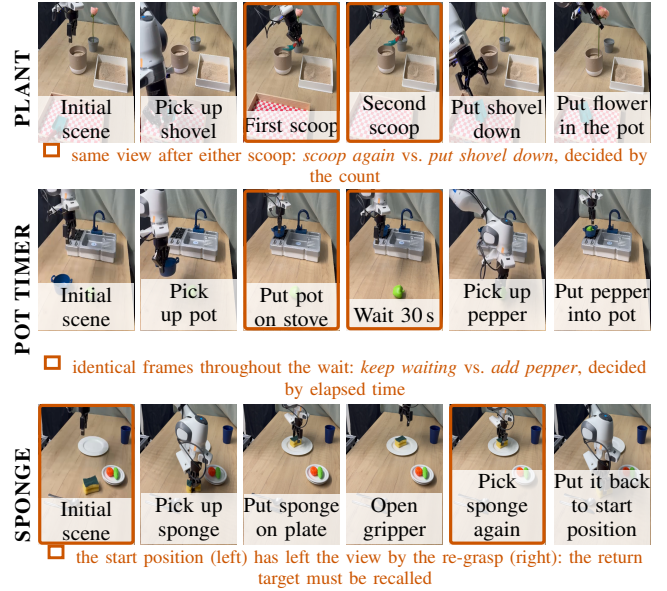


Fig. 3: Real-robot tasks, one per row, each isolating one memory demand: **PLANT** counts scoops across five stages, **POT TIMER** times an interval while the scene is static, and **SPONGE** recalls an unmarked start position that has left the camera view. Outlined frames mark the decision point where the correct action depends on an earlier event, not on the current observation.

is largely Markovian, it tests whether full-history recurrence preserves performance when the current observation is sufficient. We use LIBERO-Plus [38] to evaluate robustness to controlled visual and physical perturbations. RMBench [39] contains nine long-horizon bimanual tasks: five $M(1)$ and four $M(n)$ in which actions depend on past events no longer observable. Following its protocol, we train one policy per task on 50 demonstrations and evaluate 100 rollouts with unseen scenes.

b) Real robot setup: We further evaluate the policies on three real-robot tasks that probe distinct memory demands under real-world execution variability. Unlike simulation, real-world manipulation introduces visual clutter, contact variation, friction, and trajectory variability, which can interfere with retaining task-relevant history. We therefore consider three tasks requiring *spatial recall*, *discrete event counting*, and *interval timing*, each containing states where the correct action cannot be determined from the current observation alone. The three tasks are shown in Fig. 3. We use a single-arm Franka Emika Panda in the DROID setup [40]. The robot receives 224×224 RGB images from a fixed right-side camera and a wrist-mounted camera, together with joint-space proprioception. Demonstrations are collected through teleoperation at 15 Hz. The policy predicts an eight-dimensional action comprising seven joint positions and a continuous gripper width.

The **SPONGE** task evaluates spatial recall under clutter and broad spatial variation. The sponge is initialized across the robot’s reachable workspace without any positional markers

indicating its start location. The robot places it on a plate, releases it, and must then return it to its original position, which is no longer directly observable once the sponge has moved. This requires the remembered location to be maintained through the intervening manipulation and surrounding clutter. 48 demonstrations, ~ 480 steps per episode.

The **PLANT** task evaluates discrete event counting within a long-horizon, multi-stage manipulation sequence. The robot must place exactly two scoops of soil into a pot, put down the shovel, and pick up and insert a flower into the pot. Since the observations during the two scooping cycles are visually similar, the policy must track the number of completed scoops to determine whether to repeat the scooping action or proceed to the remaining stages. 45 demonstrations, $\sim 1,200$ steps per episode, with varying scoop durations (125.4 ± 24.3 frames).

The **POT TIMER** task evaluates interval timing. The robot places a pot on a stove, waits for approximately 30 seconds, and then picks up a pepper and places it in the pot. The visual scene remains nearly unchanged during the waiting period and provides no direct indication of elapsed time, requiring the policy to maintain temporal state internally. 45 demonstrations, ~ 980 steps per episode.

c) *Protocol*: We train one policy per task and configuration with a single seed for 500 epochs and evaluate the final checkpoint. Each policy runs 20 rollouts per task with randomized initial object positions. We report final success and the stage each rollout reached. In SPONGE, a return counts as success when the sponge lands within 2-4 cm of its original centre. In POT TIMER, a wait counts as correct when it lasts 29-31 s. With 20 rollouts, one rollout is five percentage points, and differences of that size are within rollout noise.

d) *Metrics*: We report *success rate* in all evaluations. For the real robot we also report *policy inference latency* and its reciprocal rate (Table VII), and *trajectory smoothness* as the average spectral arc length (SPARC) [41] of the joint velocity profiles across all joints. Higher SPARC is smoother.

B. Baselines

For LIBERO and RMBench we report results from prior work. For the real-robot tasks we retrained every baseline on our demonstrations with the same cameras, control rate, action space, and rollout protocol. The short-history variants are our own extensions, since neither base method ships one. The baselines cover three ways a policy can access the past. All baselines run on the robot’s inference GPU 4060Ti. This excludes memory-augmented VLAs built on multi-billion-parameter backbones [1]. An official implementation of DSSP [10] was not available at the time of our experiments, so we discuss it as concurrent work and restrict comparisons to the methods above and to reported benchmark results. The baselines cover three ways a policy can access the past. (i) **No memory**: **DP** is a standard Diffusion Policy conditioned on the current right- and wrist-view images and proprioception, and **X-VLA** [42] is a vision-language-action policy with no persistent temporal memory. (ii) **Short-term memory**: **DP-H** and **X-VLA-H** add the last eight observations (stride 10), encoded by a shared trainable ResNet-18 and by

a frozen Florence-2 encoder compressing them into eight history tokens through a two-layer Transformer, respectively. (iii) **Explicit attention-based memory**: **Gated Memory Policy (GMP)** [17] retrieves visual and action information from a finite stored history using cross-attention and a calibrated binary memory gate. Diffusion Policy is the standard imitation-learning policy at a size comparable to ours (268M parameters). X-VLA is a 0.9B-parameter pretrained VLA and one of the strongest model that fits the robot’s GPU. GMP is the closest published memory policy at comparable size. All baselines were selected to be deployable on the robot’s inference GPU, which excludes recent memory-augmented VLAs built on multi-billion-parameter backbones [1].

C. Results and Discussion

Unless stated otherwise, ReCAT denotes the Mamba-2 instantiation.

RQ1: ReCAT is competitive on standard manipulation and strong on memory-dependent manipulation. On LIBERO, ReCAT reaches 95.3% average success across the four suites (Table II). Diffusion Policy reaches 72.4%. X-VLA, which uses large-scale pretraining, reaches 98.1%. On LIBERO-Plus, ReCAT reaches 64.2% and X-VLA 71.4%. ReCAT therefore remains competitive on tasks that the current observation alone can solve.

On RMBench, ReCAT reaches 62.4% average success over nine tasks (Table III). It has the best or tied-best score on six of them, including three of the four $M(n)$ tasks. EventVLA has a higher overall average of 67.9%. It builds on a 4B-parameter backbone, about ten times larger than ReCAT, with an additional large-scale pretraining phase. Performance varies across tasks. ReCAT reaches 100% on Put Back Block and Swap Blocks but 4% on Observe and Pick Up and 6% on Press Button. Chronos [11] is not in the table because we could not establish a matched protocol as it is not evaluated on all tasks.

The real-robot tasks show the largest gap (Table IV). Policies without persistent memory succeed in 0-8.3% of rollouts. X-VLA-H, the strongest baseline, reaches 8.3% (95% Wilson CI [3.6%, 18.1%]). ReCAT variants reach 50.0-66.7%, and the best variant reaches 66.7% (95% CI [54.1%, 77.3%]). The two intervals do not overlap. All three ReCAT variants wait the correct interval in every POT TIMER rollout, and the Mamba variants complete 85-90% of PLANT rollouts. The stage-wise columns of Table IV show where the baselines fail, which we examine next.

Stage-wise view. The three tasks share a structure: observation-driven stages lead to one decision that the current observation cannot resolve. Table IV breaks each rollout down by stage. Baselines complete the early stages and fail at that decision. X-VLA-H places the sponge on the plate in 15 of 20 rollouts but returns it to the right place in only 2. In PLANT, it scoops at least once in 19 of 20 rollouts but completes the task in 2, and in 17 of them it keeps scooping past two. In POT TIMER, it places the pot in every rollout but waits the correct interval in 1. ReCAT completes the decision in

TABLE II: Success rates (%) on LIBERO.

Method	Spatial	Object	Goal	LIBERO-10	Avg.	LIBERO-Plus
Diffusion policy	78.3	92.5	68.3	50.5	72.4	—
X-VLA [42]	98.2	98.6	97.8	97.6	98.1	71.4
Ours	96.4	97.2	97.2	90.4	95.3	64.2

TABLE III: Success rates (%) on RMBench. * Indicates models utilizing large scale pretraining.

Task	TMC	DP (90M)	ACT (80M)	*p10.5 (3B)	*X-VLA (0.9B)	MEM-0 (10B)	EventVLA (4B)	*MemoryVLA (7.3B)	*MemER (10B)	Ours (374M)
Observe and Pick Up	$M(1)$	1	1	9	9	4	21	2	7	4
Rearrange Blocks	$M(1)$	0	29	13	13	89	96	53	17	96
Put Back Block	$M(1)$	0	0	11	18	90	95	81	0	100
Swap Blocks	$M(1)$	11	2	24	16	67	96	76	14	100
Swap T	$M(1)$	20	2	15	3	14	87	9	7	64
Average	$M(1)$	6.4	6.8	14.4	11.8	52.8	79.0	44.2	9.0	72.8
Battery Try	$M(n)$	10	19	16	26	28	35	33	27	43
Blocks Ranking Try	$M(n)$	10	0	6	1	18	81	53	12	99
Cover Blocks	$M(n)$	0	0	0	2	68	97	69	40	50
Press Button	$M(n)$	0	0	0	0	0	3	0	0	6
Average	$M(n)$	5.0	4.8	5.5	7.3	28.5	54.0	38.8	19.8	49.5
Overall average	—	5.8	5.9	10.4	9.8	42.0	67.9	41.8	13.8	62.4

most rollouts, and the failures that remain, occur before the temporal memory decision stage.

RQ2: Both the observation encoder and the memory conditioning matter, and the largest effects appear on PLANT.

Observation encoder. We fix the memory to Mamba-2 and vary the frame encoder (Table VI). The hybrid Mamba-attention encoder reaches 66.7% average success. A full-Transformer encoder reaches 25.0%. It drops from 85% to 0% on PLANT, from 95% to 65% on POT TIMER, and from 20% to 10% on SPONGE. A pure-SSM configuration removes the attention layer from both the frame encoder and the memory. It reaches 25% on PLANT, the only task on which we evaluated it. Removing the memory module gives 0% on all three tasks. Removing the multimodal encoder also gives 0%. The encoder supplies both the decoder input and the memory input, so these differences are end-to-end effects of representation quality. Counting is the most sensitive demand: repeated scoops must produce consistent representations before the memory can accumulate them. The no-memory result shows that the temporal module as a whole is necessary. It does not separate the contributions of its recurrent layers and its attention layer.

Memory conditioning. We fix the encoder and memory and vary how m_t enters the four-block decoder (Table V). Memory cross-attention in every block reaches 66.7% average success. Cross-attention in the final two blocks only (late fusion) reaches 38.3%. Modulation through AdaLN, a learned scale, or a content gate reaches 31.7%, 28.3%, and 8.3%. Memory dropout changes training rather than the readout and reaches 20.0%. The largest gap is on PLANT, where every-block cross-attention reaches 85% and no alternative exceeds 10%. Several alternatives fail before the count matters. Late fusion never picks up the shovel. Dropout and Scale miss it in 17 and 15 of 20 rollouts. The Gate and AdaLN variants stop after a single scoop in 7 of 20 rollouts each. Weak conditioning therefore harms both manipulation and the count readout. In these evaluations, POT TIMER is less sensitive. Every variant

that places the pot also waits the correct interval. Late fusion and AdaLN reach 90%, against 95% for every-block cross-attention. The Gate variant places the pot in only 10 of 20 rollouts, and in 6 of the other 10 it reaches for the pepper first. SPONGE is the exception. Late fusion reaches the plate in 14 of 20 rollouts and returns the sponge correctly in 5 of them, for the highest observed success at 25%. Every-block cross-attention reaches the plate in 4 of 20 and returns all 4, for 20%. That difference is one rollout. No evaluated variant reaches high SPONGE success. Grasping and spatial recall must both be learned from 48 demonstrations with broad position variation, and conditioning choices do not close that gap.

RQ3: The update rule and the memory capacity have task-dependent effects, and the additive rules have the highest observed success on counting.

Update rule. We keep the encoder, the attention layer, and the decoder fixed and replace the recurrent update (Table IV). On PLANT, Mamba-2 and Mamba-3 reach 85% and 90%. Gated DeltaNet-2 reaches 20%. It records two scoops in 14 of 20 rollouts but completes the task in 4, so most of its failures come after the count is reached. On POT TIMER, all three rules wait the correct interval in every rollout and reach 90-100% success. On SPONGE, Gated DeltaNet-2 has the highest observed success at 40%. Mamba-2 reaches 20% and Mamba-3 0%. The stage-wise columns qualify this gap. Mamba-2 and Gated DeltaNet-2 return the sponge correctly in every rollout that reaches the plate, 4 of 4 and 8 of 8. Mamba-3 never reaches the plate. The end-to-end difference on SPONGE therefore arises before the recall step, in grasping and placement. The update rules motivate a hypothesis for the PLANT pattern. Additive writes let evidence of repeated scoops accumulate, while the delta rule replaces the content stored under a matching cue and may represent multiplicity less directly (Table I). We do not measure the internal count, so this remains a hypothesis. Mamba-2 has the highest average success and is the default in all other experiments.

Memory capacity. We fix the rule to Mamba-2 and vary depth and state width, with PLANT as the most sensitive task. Reducing the memory from six blocks to two, which keeps the attention layer and leaves one Mamba-2 layer, lowers PLANT success from 85% to 65%. For state width we evaluate $d_{\text{state}} \in \{32, 64, 128, 256\}$ with everything else fixed. PLANT success is 15%, 20%, 85%, and 0%. At widths 32 and 64, 35-40% of rollouts reach two scoops but do not complete the task due to precision failures. At width 256 the policy fails before reaching two scoops. Width 128 has the highest observed success in this sweep. The non-monotonic result shows sensitivity to the configuration under a single training run. It does not establish that larger states hold less. We use six blocks and width 128 in all other experiments.

Computational efficiency. Table VII compares model size, inference cost, and trajectory smoothness under the same hardware and inference settings. ReCAT has 374M parameters, of which 142M are trainable. Policy inference takes 59 ms per step, against 110 ms for DP-H, 93 ms for GMP, and 288 ms for X-VLA-H. The reciprocal rate is 16.9 Hz. ReCAT also

TABLE IV: Real-robot results, step by step. Each task moves from observation-driven stages to a later decision that depends on retained history (outlined in Fig. 3). Breaking results down this way, instead of reporting only final success, shows exactly where a policy fails. For PLANT, the three scoop columns are mutually exclusive: a rollout is counted under the number of scoops it performed before it either moved on to the flower or stayed in the scooping loop until timeout. For POT TIMER, “No wait” and “Wrong wait” are rollouts that left the waiting state early or outside the 29-31 s window. For SPONGE, “Returned to wrong position” and “Returned to same position” partition the rollouts that placed the sponge, except rollouts that stalled at the plate. Success is the final column of each task. All values are percentages of 20 rollouts with randomised start positions. Avg. is the mean over the success rate of three tasks.

Method	Plant				Pot timer						Sponge			Avg.
	1 scoop	2 scoops	3 or more	Flower planted Success	Pot on stove	No wait	Wrong wait	≈30 s wait	Pepper in the pot	Success	Pick sponge, put on plate	Returned to wrong position	Returned to same position Success	
Baselines														
Diffusion policy [43]	–	–	–	0	–	–	–	–	–	0	–	–	0	0.0
Diffusion policy [43] + history	15	10	0	0	100	25	0	0	25	0	15	–	0	0.0
X-VLA [42]	–	–	–	0	–	–	–	–	–	0	–	–	0	0.0
X-VLA [42] + history	0	10	85	10	100	0	35	5	40	5	75	65	10	8.3
Gated Memory Policy (GMP) [17]	65	20	10	15	95	20	10	5	35	5	50	50	0	6.7
Ours														
ReCAT-Mamba-2	0	100	0	85	100	0	0	100	95	95	20	0	20	66.7
ReCAT-Mamba-3	0	95	0	90	100	0	0	100	100	100	0	0	0	63.3
ReCAT-Gated DeltaNet 2	0	70	0	20	100	0	0	100	90	90	40	0	40	50.0

TABLE V: Step-wise real-robot success across six memory-to-action conditioning mechanisms. Columns as in Table IV.

Integration	Plant				Pot timer						Sponge			Avg.
	1 scoop	2 scoops	3 or more	Flower planted Success	Pot on stove	No wait	Wrong wait	≈ 30 s wait	Pepper in the pot	Success	Pick sponge, put on plate	Returned to wrong position	Returned to same position	
ReCAT, cross-attention	0	100	0	85	100	0	0	100	95	95	20	0	20	66.7
Integration variants														
Late fusion	0	0	0	0	90	0	0	90	90	90	70 [†]	40	25	38.3
AdaLN	35	15	0	5	100	0	0	100	90	90	25	5	0	31.7
Scale	0	25	0	10	80	0	0	80	75	75	40	40	0	28.3
Dropout	5	10	0	0	90	0	0	90	60	60	20	10	0	20.0
Gate	35	5	0	5	50	0	0	50	20	20	20	0	0	8.3

TABLE VI: Real-robot success rates for observation-encoder and component ablations.

Setting	Sponge	Plant	Pot timer	Avg.
Ours, hybrid encoder	20	85	95	66.7
Ours, full-transformer encoder	10	0	65	25.0
Ours, pure-SSM encoder	–	25	–	–
Ours, w/o multimodal observation encoder tokens	0	0	0	0.0
Ours, without memory module	0	0	0	0.0

TABLE VII: Computational cost on the same hardware and inference setting at. Latency is per policy inference; the rate is its reciprocal. SPARC: higher is smoother.

Metric	ReCAT	DP-H	X-VLA-H	GMP
Total / trainable parameters	374M / 142M	267.9M / 267.9M	881.9M / 881.9M	282.2M / 188.7M
Peak inference VRAM	1.44GB	0.65GB	2.55GB	1.09GB
Inference latency/frequency	0.059s/16.9hz	0.11s/9hz	0.288s/3.47hz	0.093s/10.75hz
Smoothness metric (SPARC [41])	-5.16	-6.19	-6.76	-13.06

has the highest SPARC, that is the smoothest end-effector trajectories, among the compared policies.

V. CONCLUSIONS

We presented ReCAT, a structured recurrent policy for memory-dependent manipulation. It’s memory integrates the full observation history through Mamba-2 layers and one causal attention layer. It’s flow-matching decoder reads the current observation and the memory through separate cross-attention in every block. This design reaches 95.3% on

LIBERO, 62.4% on RMBench, and 66.7% across three real-robot tasks that require spatial recall, counting, and timing. It also has the lowest measured inference latency among the compared memory-based policies, at 59 ms per step. The analysis explains where this performance comes from. Stage-wise results show that short-history policies execute the immediate manipulation but fail at the decision that depends on earlier information. Within the evaluated configurations, the hybrid observation encoder and memory cross-attention in every decoder block gave the highest average success. The recurrent update rule has task-dependent effects: the additive rules have the highest observed success on counting and timing, and the delta rule on spatial recall. Effective memory-dependent control therefore, depends on what is written to memory, how it is updated, and how the action generator reads it alongside the current observation.

VI. LIMITATIONS AND FUTURE WORK

Our real-robot evaluation covers one task per memory demand, each evaluated over 20 rollouts. Broader studies across multiple tasks, embodiments, are needed to assess generalization, including multiple counts and durations distinguished by language instructions. The memory ablation removes the recurrent layers and the attention layer together, so their individual contributions remain open. The SPONGE results show a remaining trade-off between memory use and fine-grained spatial control. Future work should examine these

factors and improve success on spatial-return manipulation under limited demonstrations with larger task diversity.

ACKNOWLEDGMENT

We used Claude and ChatGPT to polish the text of this paper. The authors reviewed every generated item, and all technical claims, experiments, and results are our own. The research presented in this paper was funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – 448648559. The authors gratefully acknowledge the computing time provided on the high-performance computer HoreKa by the National High-Performance Computing Center at KIT and by Gauss Centre for Supercomputing e.V. (www.gauss-centre.eu) for GCS Supercomputer JUPITER at Jülich Supercomputing Centre (JSC).

REFERENCES

- [1] H. Shi, B. Xie, Y. Liu, L. Sun, F. Liu, T. Wang, E. Zhou, H. Fan, X. Zhang, and G. Huang, “Memoryvla: Perceptual-cognitive memory in vision-language-action models for robotic manipulation,” in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 18567–18602.
- [2] H. Li, F. Shen, D. Chen, L. Yang, X. Wang, J. Shi, Z. Bing, Z. Liu, and A. Knoll, “Remem-vla: Empowering vision-language-action model with memory via dual-level recurrent queries,” *arXiv preprint arXiv:2603.12942*, 2026.
- [3] A. Gu, K. Goel, and C. Re, “Efficiently modeling long sequences with structured state spaces,” in *International Conference on Learning Representations*, 2021.
- [4] S. Yang, J. Kautz, and A. Hatamizadeh, “Gated delta networks: Improving Mamba2 with delta rule,” *arXiv preprint arXiv:2412.06464*, 2024.
- [5] A. S. Lahoti, K. Li, B. Chen, C. Wang, A. Bick, Z. Kolter, T. Dao, and A. Gu, “Mamba-3: Improved sequence modeling using state space principles,” in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 86173–86202.
- [6] P. Vanjani, Z. Li, J. Suliga, M. Reuss, G. Geraci, X. Jiang, and R. Lioutikov, “Dam-vla: Decoupled asynchronous multimodal vision language action model,” *arXiv preprint arXiv:2606.12105*, 2026.
- [7] Y. Zhou, Y. Lin, F. Peng, J. Chen, K. Huang, H. Yang, and Z. Yin, “Mtil: Encoding full history with mamba for temporal imitation learning,” *IEEE Robotics and Automation Letters*, 2025.
- [8] Y. Yoo, J. Hu, Y. Zhu, B. Liu, Q. Liu, R. Martín-Martín, and P. Stone, “Robossm: Scalable in-context imitation learning via state-space models,” *arXiv preprint arXiv:2509.19658*, 2025.
- [9] T. Tsuji, “Mamba as a motion encoder for robotic imitation learning,” *IEEE Access*, vol. 13, pp. 69941–69949, 2025.
- [10] Z. Guan, J. Hu, H. Fang, Y. Jiang, Y. Huang, S. Li, X. Li, and Y. Ban, “Dssp: Diffusion state space policy with full-history encoding,” *arXiv preprint arXiv:2605.14598*, 2026.
- [11] Y. Zhou, Y. Wang, N. Wang, S. Xing, S. Tu, X. Li, J. Zhang, N. Jiang, Y. Lin, H. Yang, *et al.*, “Chronos: A physics-informed full-history framework for non-markovian long-horizon manipulation,” *arXiv preprint arXiv:2606.30318*, 2026.
- [12] E. Cherepanov, N. Kachaev, D. Zelezetsky, A. Bulatov, A. Pshenitsyn, Y. Kuratov, A. Skrynnik, A. I. Panov, and A. K. Kovalev, “ μ vla: On recurrent memory for partially observable manipulation in vla models,” *arXiv preprint arXiv:2606.12497*, 2026.
- [13] M. S. Mark, J. Liang, M. Attarian, C. Fu, D. Dwibedi, D. Shah, and A. Kumar, “Bpp: Long-context robot imitation learning by focusing on key history frames,” *arXiv preprint arXiv:2602.15010*, 2026.
- [14] C. Yin, W. Xu, J. Yang, S. Tam, H. Liu, Y. Yao, X. Zeng, J. Cui, Y. Wang, Z. Yin, *et al.*, “Simplememvla: A simple but effective native-video memory for vision-language-action models,” *arXiv preprint arXiv:2609.05533*, 2026.
- [15] M. Torne, K. Pertsch, H. Walke, K. Vedder, S. Nair, B. Ichter, A. Z. Ren, H. Wang, J. Tang, K. Stachowicz, *et al.*, “Mem: Multi-scale embodied memory for vision language action models,” *arXiv preprint arXiv:2603.03596*, 2026.
- [16] R. Shah, Y. Li, F. Bello, Y. Zhu, and R. Martín-Martín, “Memory retrieval in visuomotor policies for long-horizon robot control,” *arXiv preprint arXiv:2606.25136*, 2026.
- [17] Y. Gao, J. Liu, S. Li, and S. Song, “Gated memory policy,” *arXiv preprint arXiv:2604.18933*, 2026.
- [18] X. Jia, Q. Wang, A. Donat, B. Xing, G. Li, H. Zhou, O. Celik, D. Blessing, R. Lioutikov, and G. Neumann, “Mail: Improving imitation learning with selective state space models,” in *8th Annual Conference on Robot Learning*, 2024.
- [19] J. Cao, Q. Zhang, J. Sun, J. Wang, H. Cheng, Y. Li, J. Ma, K. Wu, Z. Xu, Y. Shao, *et al.*, “Mamba policy: Towards efficient 3d diffusion policy with hybrid selective state models,” in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2025, pp. 11359–11366.
- [20] N. Oh, J. Jang, M. Jung, and D. Park, “Dispo: diffusion-ssm based policy learning for coarse-to-fine action discretization,” *arXiv preprint arXiv:2409.14719*, 2024.
- [21] Y. Takagi, M. Kambara, D. Yashima, K. Seno, K. Tokura, and K. Sugiura, “Anolevla: Lightweight vision-language-action model with deep state space models for mobile manipulation,” *arXiv preprint arXiv:2603.15046*, 2026.
- [22] Y. Jiang, Y. Chebotar, R. Zheng, F. Hu, Y. Ge, J. Wu, T. Dai, S. Reed, L. Fei-Fei, Y. Zhu, *et al.*, “Robottt: Context scaling for robot policies,” *arXiv preprint arXiv:2607.15275*, 2026.
- [23] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret, “Transformers are RNNs: Fast autoregressive transformers with linear attention,” in *International Conference on Machine Learning*. PMLR, 2020, pp. 5156–5165.
- [24] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv preprint arXiv:2312.00752*, 2023.
- [25] T. Dao and A. Gu, “Transformers are ssms: Generalized models and efficient algorithms through structured state space duality,” *arXiv preprint arXiv:2405.21060*, 2024.
- [26] I. Schlag, K. Irie, and J. Schmidhuber, “Linear transformers are secretly fast weight programmers,” in *International Conference on Machine Learning*. PMLR, 2021, pp. 9355–9366.
- [27] S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim, “Parallelizing linear transformers with the delta rule over sequence length,” *Advances in neural information processing systems*, vol. 37, pp. 115491–115522, 2024.
- [28] S. Yang, J. Kautz, and A. Hatamizadeh, “Gated delta networks: Improving mamba2 with delta rule,” in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 29687–29707.
- [29] R. Grazzi, J. Siems, A. Zela, J. Franke, F. Hutter, *et al.*, “Unlocking state-tracking in linear rnns through negative eigenvalues,” in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 36565–36597.
- [30] S. Arora, S. Eyuboglu, A. Timalsina, I. Johnson, M. Poli, J. Zou, A. Rudra, and C. Ré, “Zoology: Measuring and improving recall in efficient language models,” *arXiv preprint arXiv:2312.04927*, 2023.
- [31] K. Team, Y. Zhang, Z. Lin, X. Yao, J. Hu, F. Meng, C. Liu, X. Men, S. Yang, Z. Li, *et al.*, “Kimi linear: An expressive, efficient attention architecture,” *arXiv preprint arXiv:2510.26692*, 2025.
- [32] J. Lee, S. Lee, J. Shin, H. Jung, S. Kim, D. Cho, H. J. Kim, J.-B. Huang, and F. Huang, “Dynaflip: Rethinking robotics perception via tri-modal-dynamics guided representation,” *arXiv preprint arXiv:2605.30350*, 2026.
- [33] J. Gao, F. Ye, J. Zhang, and W. Qian, “Compressor-vla: Instruction-guided visual token compression for efficient robotic manipulation,” *arXiv preprint arXiv:2511.18950*, 2025.
- [34] M. Reuss, H. Zhou, M. Rühle, Ömer Erdinç Yağmurlu, F. Otto, and R. Lioutikov, “Flower: Democratizing generalist robot policies with efficient vision-language-action flow policies,” 2025. [Online]. Available: <https://arxiv.org/abs/2509.04996>
- [35] P. Vanjani, P. Mattes, X. Jia, V. Dave, and R. Lioutikov, “Disdp: Robust imitation learning via disentangled diffusion policies,” in *Reinforcement Learning Conference*, 2025.
- [36] M. Reuss, M. Li, X. Jia, and R. Lioutikov, “Goal-conditioned imitation learning using score-based diffusion policies,” *arXiv preprint arXiv:2304.02532*, 2023.
- [37] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “Libero: Benchmarking knowledge transfer for lifelong robot learning,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 44776–44791, 2023.

- [38] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, *et al.*, “Libero-plus: In-depth robustness analysis of vision-language-action models,” *arXiv preprint arXiv:2510.13626*, 2025.
- [39] T. Chen, Y. Wang, M. Li, Y. Qin, H. Shi, Z. Li, Y. Hu, Y. Zhang, K. Wang, Y. Chen, *et al.*, “Rmbench: Memory-dependent robotic manipulation benchmark with insights into policy design,” *arXiv preprint arXiv:2603.01229*, 2026.
- [40] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, *et al.*, “Droid: A large-scale in-the-wild robot manipulation dataset,” *arXiv preprint arXiv:2403.12945*, 2024.
- [41] A. J. Scholp, M. R. Hoffman, S. P. Rosen, S. M. Abdelhalim, C. A. Jones, J. J. Jiang, and T. M. McCulloch, “Spectral arc length as a method to quantify pharyngeal high-resolution manometric curve smoothness,” *Neurogastroenterology & Motility*, vol. 33, no. 10, p. e14122, 2021.
- [42] J. Zheng, J. Li, Z. Wang, D. Liu, X. Kang, Y. Feng, Y. Zheng, J. Zou, Y. Chen, J. Zeng, *et al.*, “X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model,” in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 60 580–60 606.
- [43] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Robotics: Science and Systems (RSS)*, 2023.